Git and GitHub

What is open source?

Open source refers to software or projects whose source code is freely available for anyone to view, modify and distribute . Open source projects promote collaboration, transparency and community driven development.

What is Version Control?

A system that tracks and manages changes to a file over time allowing developers to revert to previous versions, collaborate efficiently and maintain a history of modifications.

What is Git and GitHub?

Git is a free and open-source distributed version control system that helps you track changes in your code over time. It's a powerful tool that can help you manage your code history, collaborate with others, and deploy your applications.

GitHub is a web-based hosting service for Git repositories. It provides a number of features that make it easy to collaborate on code, such as issue tracking, pull requests, and code reviews.

What is Repository?

A repository (often abbreviated as repo) is a storage location where software projects, files, or data are stored and managed.

What does Git do?

- Manage projects with **Repositories**
- **Clone** a project to work on a local copy
- Control and track changes with **Staging** and **Committing**
- **Branch** and **Merge** to allow for work on different parts and versions of a project
- **Pull** the latest version of the project to a local copy
- **Push** local updates to the main project

Git config:

The git config command is a convenience function that is used to set Git configuration values on a global or local project level. These configuration levels correspond to .gitconfig text files. Executing git config will modify a configuration text file.

1. `git config [options]`

👉 Sets the username and email address for commits.

```
git config --global user.name "codeclash"
git config --global user.email "emaple@gmail.com"
```

Git init:

The git init command creates a new Git repository. It can be used to convert an existing, unversioned project to a Git repository or initialize a new, empty repository.

2. `git init`

👉 Initialize repository as git repository.

```
git init
```

Cloning Repositories:

Cloning a repository in Git and GitHub involves creating a local copy of a remote repository on your computer. This allows you to work on the project locally, commit changes, and later push those changes back to the remote repository.

3. `git clone [url]`

👉 Clones repository from server like Github, Gitlab.

```
git clone https://github.com/imtiyazCode/example.git
```

Git status:

The git status command is used to display the current state of your working directory and staging area. It shows which files have been modified, added, or deleted, and whether these changes have been staged for the next commit. It also tells you if your branch is ahead or behind the remote repository, or if there are any uncommitted changes. This command helps you keep track of your current progress in Git.

4. git status

Checks the current status of working tree.

```
git status
```

Adding Files in Git:

To add files in Git, you first need to stage them using the git add command. This marks the files to be tracked and included in the next commit. You can add a specific file by running git add <filename>, or to add all files in the directory, you can use git add . . Once the files are staged, you commit the changes using git commit -m “Your commit message”, which saves the changes in your local repository.

5. git add [Filename(s)]

Puts the unstage files to staging area

```
git add index.html style.css script.js
git add .        // add all unstage file
```

Git commit:

The git commit command is used to save your changes to the local repository. When you commit, you’re recording a snapshot of the changes that have been staged Each commit requires a commit message that explains what changes you made, which helps keep track of the project history.

6. git commit -m [Message]

Records changes to the git repo by saving a log message to gether with a commit ID.

```
git commit -m “fixed bug”
git commit -a    // commit all file without message
```

Git remote:

The git remote command allows you to manage remote repositories in your local Git configuration. When setting up a remote, you typically provide the URL of the remote repository.

7. git remote [Options] [Variable] [URL]

👉 Connects your local repository to the remote repository over a server

```
git remote add origin https://github.com/user/repo
```

Pushing / Pulling Changes:

When you pull changes in Git, you're fetching and integrating changes from a remote repository into your local repository. This operation updates your local branch with the latest changes from the remote branch. Whereas When you push changes in Git, you're sending your local commits to a remote repository. This operation updates the remote repository with your latest changes.

8. git push [Options] [Variable] [Branch]

👉 Push the contents of your local repository to the added remote repositoty

```
git push -u origin main   // -u means upstream
```

9. git pull [Variable] [Branch] or [URL]

👉 Fetch and integrate the contents of the remote repository to your local repository

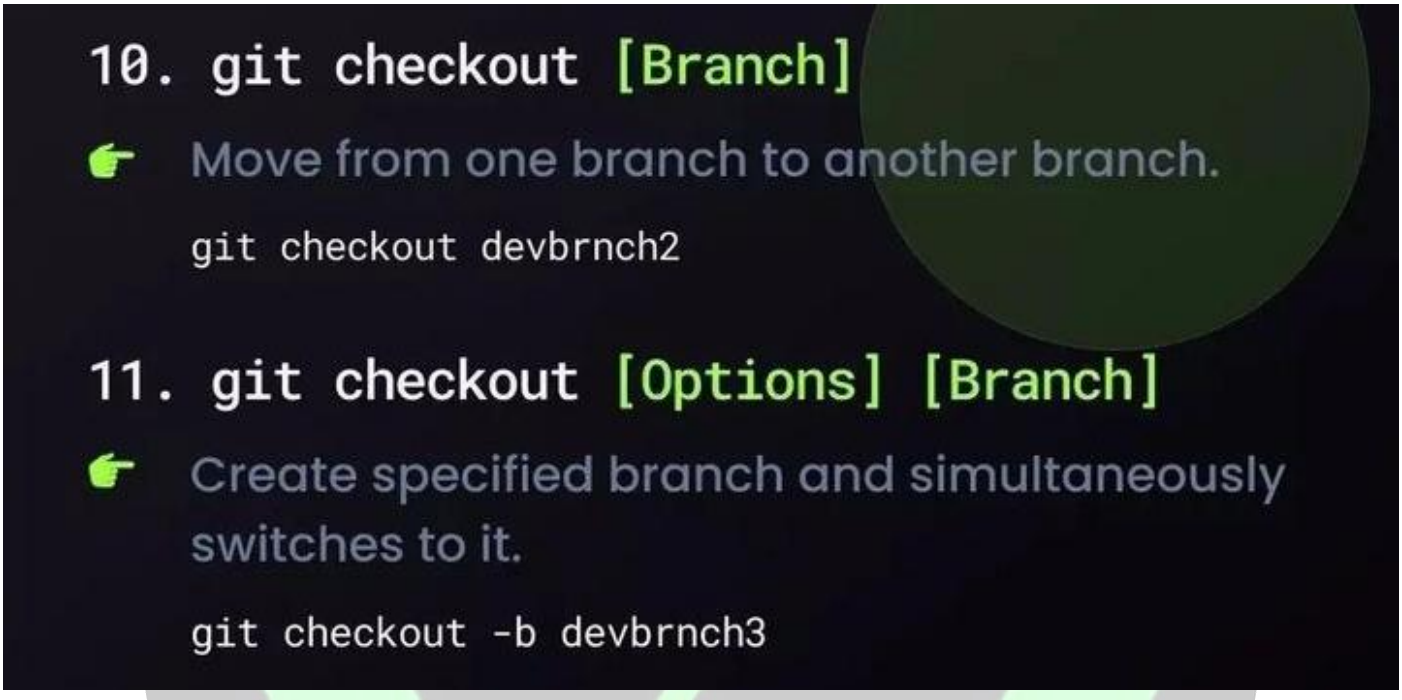
```
git pull origin main
git pull https://github.com/user/repo
```

Git Branch:

In Git, a branch is a separate line of development that allows you to work on different features or fixes without affecting the main codebase. Branches help you keep your changes isolated until you're ready to merge them back into the main branch, often called main.

Checkout Branch:

In Git, to “checkout” from a branch means to switch your working directory to that branch, making it the active branch. This updates your files to match the state of that branch and allows you to work on it.



Merging Basics:

A merge in Git is the process of combining changes from one branch into another. When you want to integrate updates from one branch (the source) into another branch (the target), you need to perform a merge. This involves resolving conflicts between the two branches, if any exist. The goal of merging is to create a new commit that represents the combined changes from both branches, resulting in a single, cohesive history for your project.

Forking vs Cloning

Forking and cloning are two fundamental concepts in Git, particularly when working with repositories hosted on platforms like GitHub, GitLab, or Bitbucket. While both actions involve copying a repository, they serve different purposes and have distinct workflows. Cloning a repository means creating a local copy of a repository that exists on a remote server (e.g., GitHub) on your local machine. This allows you to work on the project locally, make changes, and then push those changes back to the remote repository if you have the necessary permissions. Forking a repository is specific to platforms like GitHub, GitLab, and Bitbucket. When you fork a repository, you create a copy of someone else's repository in your own account. This forked repository is independent of the original and can be modified without affecting the original project.

ENIGMA